Architecture micro-services,
IA et développement full-stack

ENSET / ADRIA - Rapport des parties A à F

30 octobre 2026 - Démonstration bancaire. Rapport fondé sur les captures transmises par le candidat et sur les vérifications réellement exécutées.

Dépôt : https://github.com/ilyasouazzani/OUAZZANI_TAYBI_Ilyas_test_adria_2027

Périmètre prioritaire : A à F, à la demande du candidat. Les parties G, H et I ne sont pas présentées comme réalisées dans ce rapport. Gemini remplace OpenAI à la demande explicite du candidat.

Vérification	Résultat
Configuration centralisée	PASS
Découverte Eureka	PASS
GraphQL comptes	PASS
GraphQL transactions	PASS
Gateway	PASS
Règles métier	PASS
Kafka Docker	PASS
Outils MCP de l’agent	PASS
Réponse Gemini avec solde réel	PASS

Build validé : 31 tests automatisés sans échec, parent Maven et six modules compilés. Une validation HTTP séparée vérifie les applications ensemble. Un résultat manquant ou indisponible n’est pas remplacé par une preuve fabriquée.

A - Préparation et organisation

Le parent Maven centralise Java 17, Spring Boot 3.5.7, Spring Cloud 2025.0.0 et Spring AI 1.1.8. Il agrège account-service, transaction-service, discovery-service, config-service, gateway-service et ai-service.

Les services métier suivent les couches domain, repository, service, web/rest, web/graphql et mcp. Chaque service conserve sa propre base H2. Le fichier .env est exclu de Git ; les valeurs réelles des clés ne figurent pas dans le rapport.

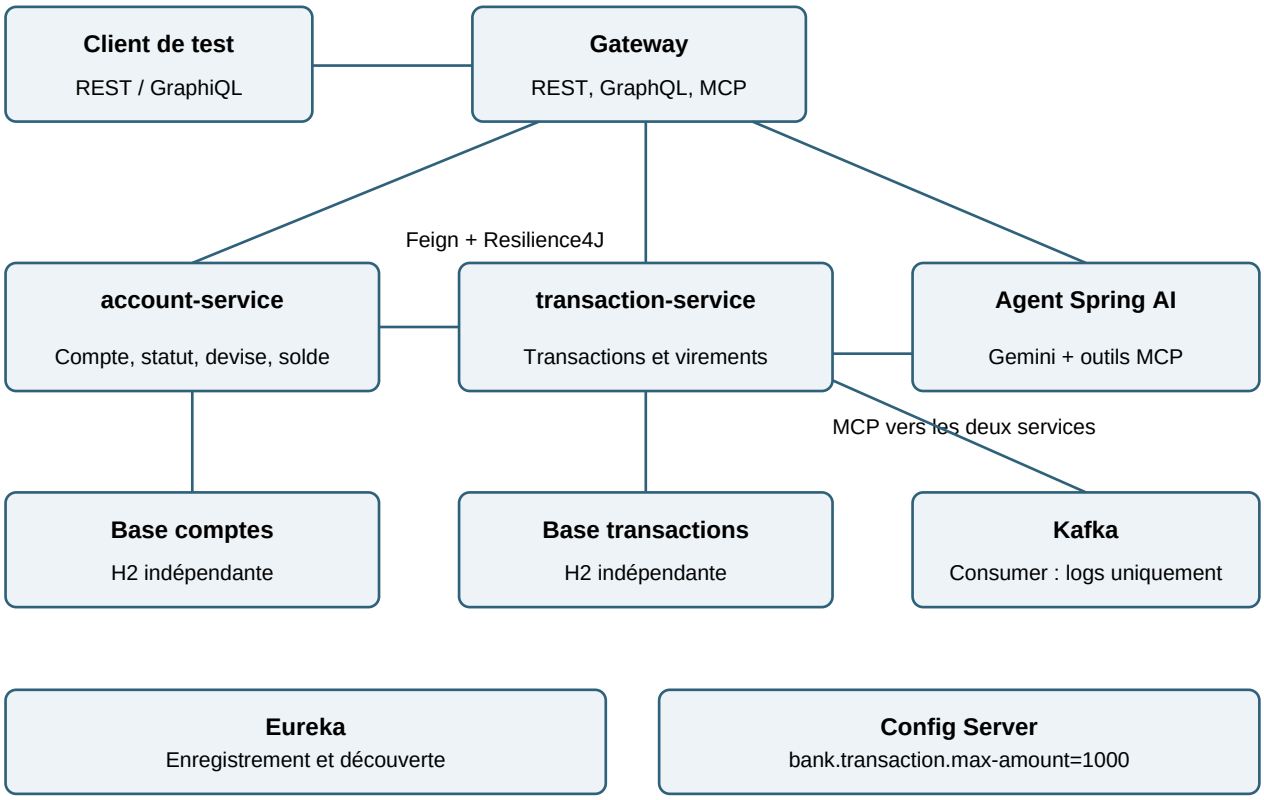
À la demande du candidat, l’agent utilise GEMINI_API_KEY à la place de la configuration OpenAI initiale. La création ou la révocation des clés s’effectue dans le compte du fournisseur ; une clé communiquée dans un chat doit être remplacée.

Le code est versionné dans le dépôt GitHub du candidat. Les secrets et les fichiers de compilation sont exclus des livrables versionnés.

Les démarches Google Sheet, visibilité finale du dépôt et dépôt Classroom doivent être vérifiées par le candidat ; aucune soumission Classroom n’a été exécutée par l’assistant.

B - Conception et architecture

Deux services métier, des bases séparées et une communication par contrats réseau. Le diagramme de classes Mermaid et le schéma d'architecture Mermaid sont livrés dans docs/.

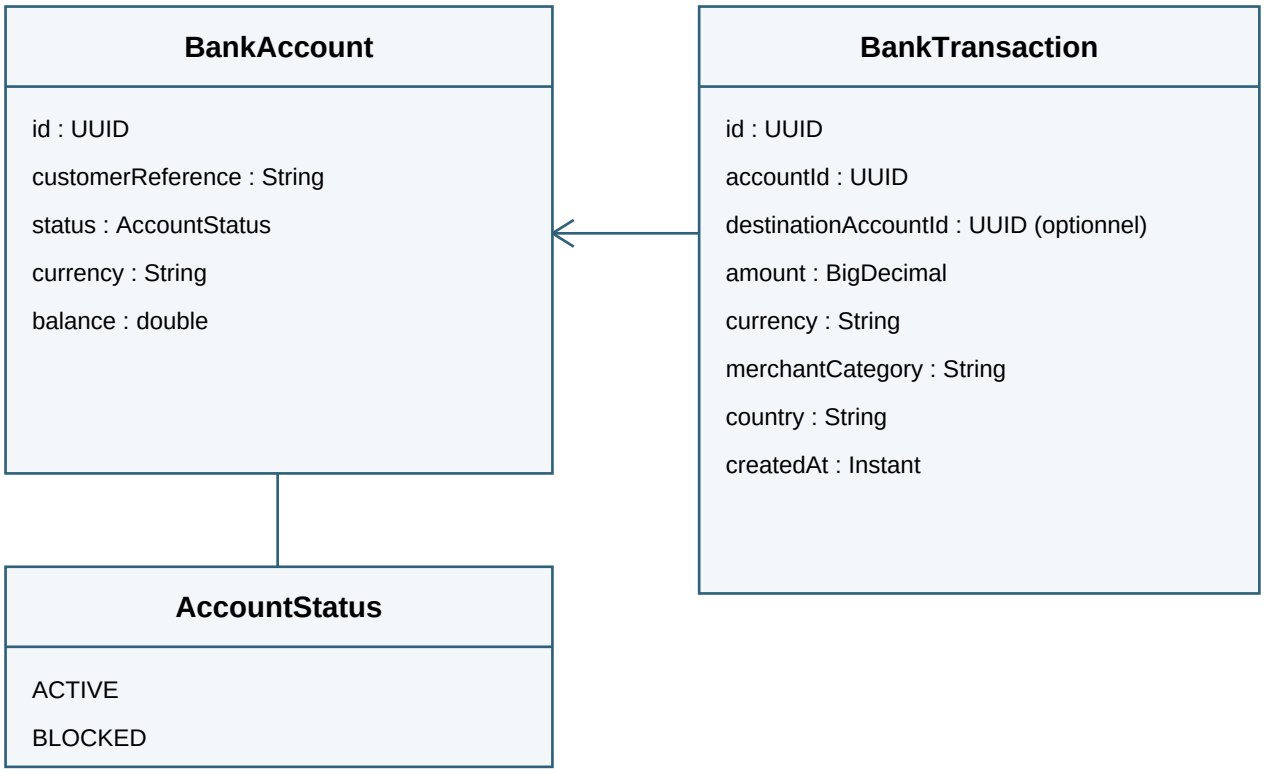


Entité	Attributs principaux
BankAccount	UUID id ; customerReference ; status ACTIVE/BLOCKED ; currency ; double balance
BankTransaction	UUID id ; accountId ; destinationAccountId optionnel ; BigDecimal amount ; currency ; merchantCategory ; country ; Instant createdAt

Les références interservices sont des UUID, sans relation JPA traversant les bases. destinationAccountId complète le modèle minimal pour identifier les virements. Les calculs intermédiaires du solde utilisent BigDecimal ; la précision limitée du double reste documentée.

B - Diagramme de classes métier

Les deux services stockent leurs propres entités. Une transaction référence les comptes par UUID. Le diagramme complet des contrôleurs, outils, services et repositories est fourni dans docs/conception.md.



La flèche entre Transaction et Account exprime une référence métier, pas une clé étrangère ou une jointure entre les bases. destinationAccountId est renseigné pour un virement. Le solde reste un double conformément au modèle minimal fourni, avec des contrôles de précision à la mise à jour.

C - Services métier et connecteurs

REST, GraphQL et MCP délèguent aux mêmes méthodes de service. La validation ne dépend donc pas seulement du contrôleur REST. Les créations et les virements appliquent les contrôles de montant positif, plafond configuré, existence et activité des comptes, devise compatible et solde suffisant.

La transaction consulte le compte via Feign, persiste son enregistrement, puis demande le débit. Un échec de l'appel entraîne le rollback local. Dans account-service, un virement verrouille les deux comptes dans un ordre stable et réalise le débit et le crédit dans une seule transaction locale.

Connecteur	Comptes	Transactions
REST	/api/accounts ; /{id}/balance	/api/transactions ; /transfers
GraphQL	/graphql ; GraphiQL /graphiql	/graphql ; GraphiQL /graphiql
MCP	/mcp ; 3 outils	/mcp ; 4 outils

Les montants d'entrée GraphQL sont des chaînes décimales, converties en BigDecimal. Les outils MCP sont listAccounts, getAccount, getAccountBalance, listAccountTransactions, getTransaction, createTransaction et transferMoney.

Resilience4J répète uniquement les lectures en échec technique transitoire, jusqu'à trois tentatives. Le circuit ouvre à partir de 50 % d'échecs, après au moins trois appels logiques dans une fenêtre de six. Après dix secondes, deux appels d'essai sont permis. Les erreurs métier 4xx ne sont pas traitées comme des pannes.

Le fallback renvoie HTTP 503 et operation not confirmed. Il ne fabrique ni solde ni succès. Les écritures ne sont pas répétées automatiquement. Le mécanisme ne garantit pas une transaction distribuée entre les bases : une réponse d'écriture perdue peut demander une réconciliation.

Captures reçues : comptes actifs, solde initial 1500 EUR, transaction 250 EUR, puis solde 1250 EUR. Les virements et le fallback disposent également de preuves de tests automatisés, mais aucune capture manuelle supplémentaire n'est inventée.

D - Composants techniques

Eureka enregistre les services et permet à OpenFeign et à Gateway de résoudre leurs noms. Config Server utilise le backend native et partage le plafond 1000 depuis les ressources classpath:/config-repo. Les services importent cette configuration au démarrage.

Composant	Port habituel	Responsabilité
Discovery	8761	Registre et dashboard Eureka
Config Server	8888	/transaction-service/default et configuration partagée
Gateway	8080	Routage vers lb://account-service, lb://transaction-service et lb://ai-service
Agent	8083	REST de conversation et liste des outils MCP

Gateway expose /api/accounts, /api/transactions et /api/assistant. Les chemins /accounts/graphql et /transactions/graphql sont réécrits vers /graphql. Les chemins /accounts/mcp et /transactions/mcp sont réécrits vers /mcp.

Les probes Actuator readiness et liveness sont activées. La configuration est chargée au démarrage ; aucun refresh dynamique n’est annoncé. Les tests HTTP finaux utilisent des ports temporaires 18xxx pour éviter des conflits avec les services déjà ouverts.

E - Agent Gemini avec outils MCP

L’agent utilise le starter Google GenAI de Spring AI et le client MCP WebFlux. Les outils des deux microservices sont découverts à distance puis transmis au ChatClient. La clé est lue via spring.ai.google.genai.api-key=${GEMINI_API_KEY:}, à partir du .env privé.

La consigne système demande de consulter les outils, de ne pas inventer de données et de demander les informations manquantes. Les contenus des outils sont traités comme des données. Une écriture ne doit être exécutée que pour une demande explicite et ne doit pas être répétée automatiquement après un échec.

GET /api/assistant/tools montre les définitions réellement découvertes. POST /api/assistant/chat accepte un message et renvoie une réponse. Un message vide ou mal formé produit HTTP 400. Un modèle désactivé ou un appel défaillant produit HTTP 503.

Question réellement envoyée : Utilise un outil pour consulter le solde du compte 175968ad-f112-416c-9c47-aef217232eb4. Réponds uniquement avec le montant du solde et sa devise.

Résultat de la découverte MCP : PASS. Résultat de génération Gemini : PASS.

Réponse HTTP observée : 200

{"answer": "1250.0 EUR"}

Le test réel valide la découverte des outils, leur appel par Gemini et une réponse cohérente avec le solde du compte. Les quatre tests automatisés de ai-service vérifient également la validation REST et les erreurs explicites.

F - Communication asynchrone Kafka

Une transaction réussie émet TransactionCreated dans la couche service. Un listener AFTER_COMMIT publie après le commit JPA sur bank.transactions.created, avec accountId comme clé. Le consumer de transaction-service journalise le topic, la clé et le payload ; il ne modifie aucune donnée bancaire.

Format minimal : eventId (UUID), eventType=TransactionCreated, transactionId (UUID), accountId (UUID), amount (nombre décimal), currency et occurredAt (date ISO-8601). Le topic local comporte trois partitions et une réplication de un. Le partitionnement par compte conserve l’affectation des événements d’un même compte à une partition.

Le test Kafka embarqué vérifie les sept champs, la clé, la réception et l’absence de modification supplémentaire en base. Un second test avec Kafka lancé dans Docker par le candidat a validé un débit MCP de 250 EUR, le passage de 1500 à 1250 EUR et la réception par le consumer.

Événement réellement reçu :

{"eventId":"fc9023ac-8a6c-475f-9333-ac892108d5bf","eventType":"TransactionCreated","transactionId":"b646aa38-c66c-4cc0-b2eb-cea743e3374b","accountId":"ca85c8ff-f174-4f76-b6a0-425f12191038","amount":250,"currency":"EUR","occurredAt":"2026-10-30T11:40:07.110704900Z"}

Limite : AFTER_COMMIT ne rend pas atomiques la base et Kafka. Un arrêt entre le commit et l’envoi ou une indisponibilité prolongée du broker peut faire manquer l’événement. Les erreurs sont journalisées ; aucune outbox persistante n’est annoncée. L’idempotence du producer ne constitue pas une garantie exactly-once de bout en bout.

Validation et limites du livrable

Le build validé compile le parent et les six modules avec 31 tests réussis. Le test Gemini connecté aux outils MCP a également réussi. Les tests HTTP finaux et les validations antérieures sont conservés dans docs/verification-*.json et docs/verification-*.txt. Les preuves mentionnent le contexte du test, notamment broker embarqué, broker Docker ou doubles de test.

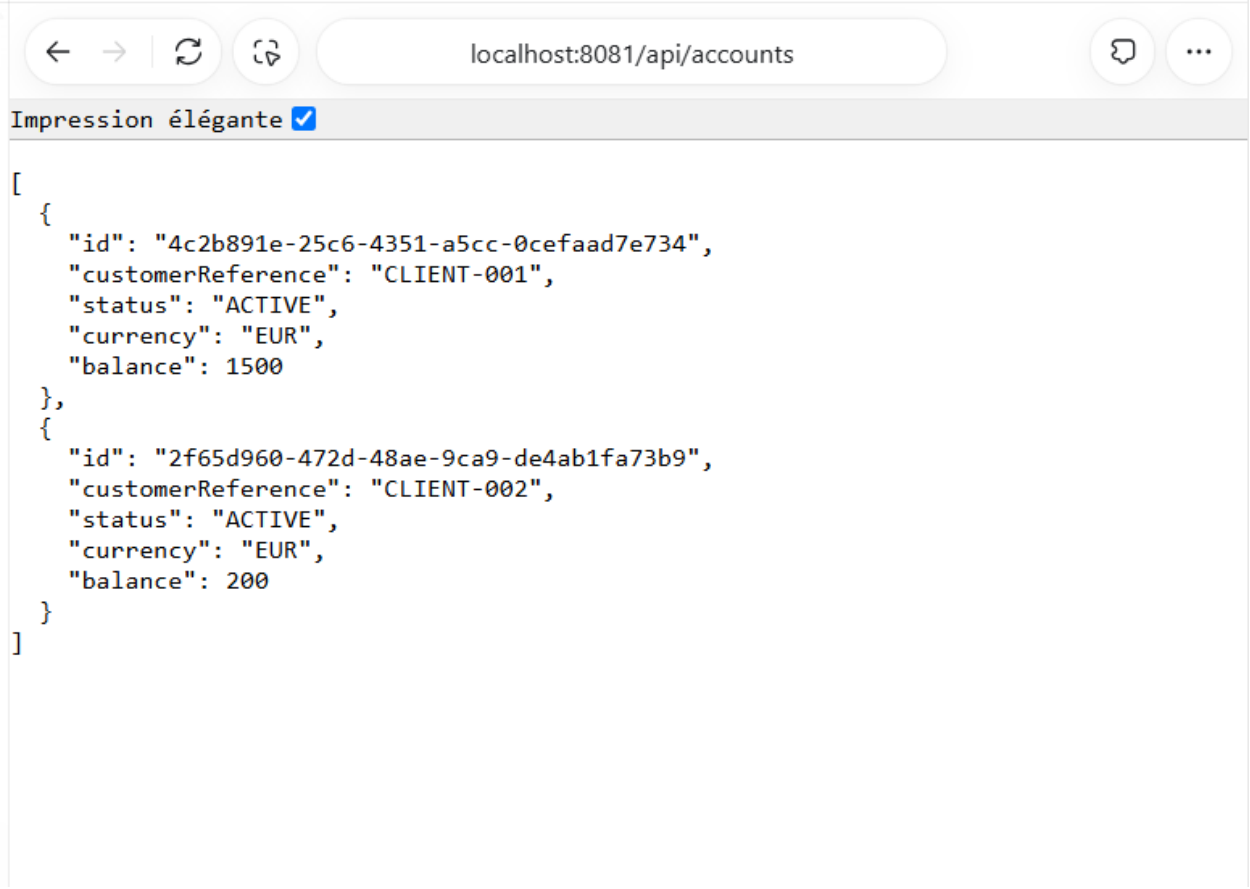
Les captures ci-après proviennent du candidat. Celles des outils MCP montrent les réponses réellement enregistrées de tools/list, affichées ensuite dans le navigateur ; ce ne sont pas des nouveaux endpoints métier. Les captures de l’agent, de GraphiQL et de Gateway restent à ajouter si elles n’ont pas encore été transmises.

Les parties G, H et I sont laissées hors du périmètre prioritaire A-F. Le compose Kafka n’est pas présenté comme le compose complet de tous les services. Aucun pipeline de déploiement, aucune soumission Classroom et aucune publication cloud ne sont déclarés réalisés.

Sources : énoncé Test ADRIA ENSET 2027.pdf fourni par le candidat ; code et résultats du dépôt ; documentation officielle Spring AI (Google GenAI et MCP), Spring Cloud et Spring GraphQL. Les divergences volontaires et limites de cohérence sont explicitées dans les sections correspondantes.

Capture - 02-01-comptes.png

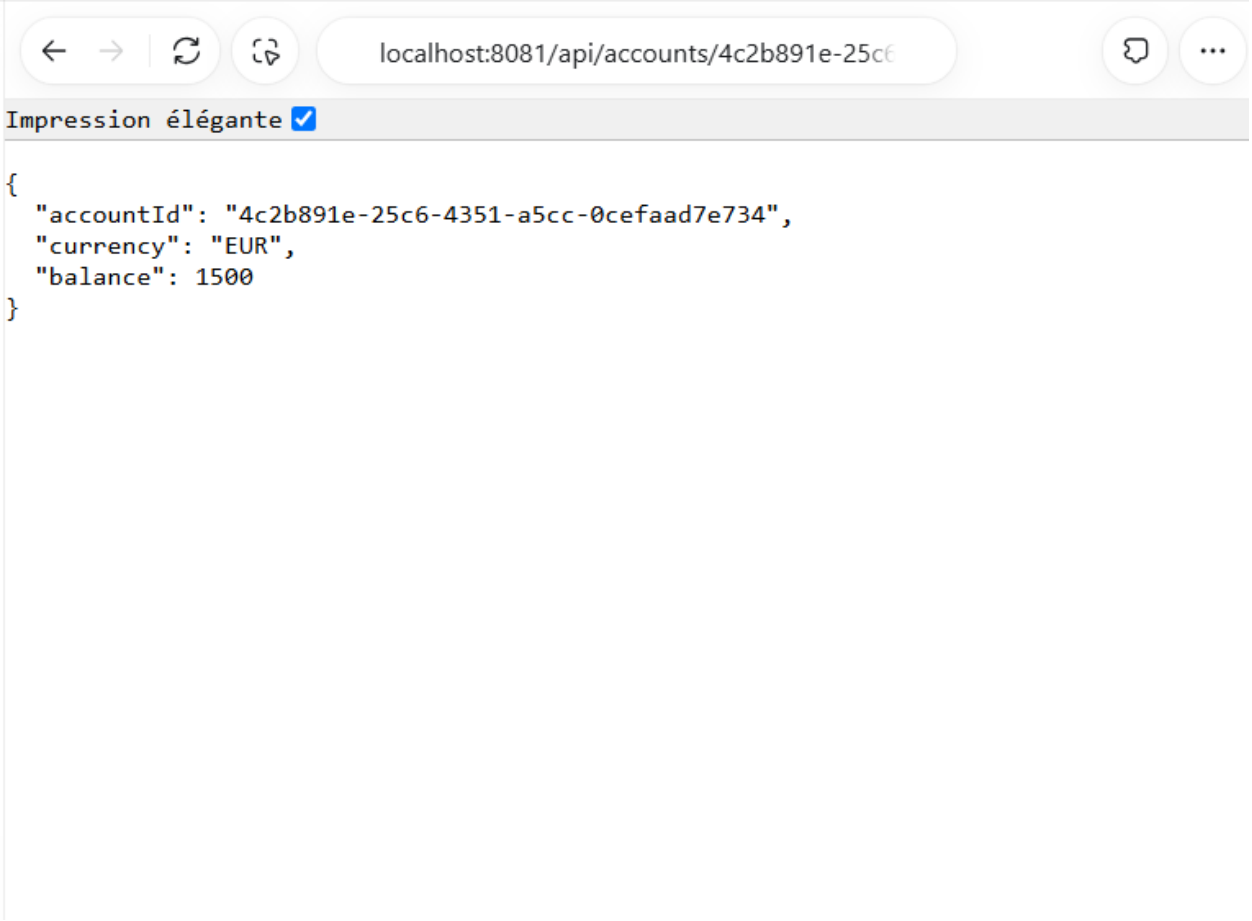
Comptes actifs et soldes initiaux : 1500 EUR et 200 EUR.




```
[
  {
    "id": "4c2b891e-25c6-4351-a5cc-0cefaad7e734",
    "customerReference": "CLIENT-001",
    "status": "ACTIVE",
    "currency": "EUR",
    "balance": 1500
  },
  {
    "id": "2f65d960-472d-48ae-9ca9-de4ab1fa73b9",
    "customerReference": "CLIENT-002",
    "status": "ACTIVE",
    "currency": "EUR",
    "balance": 200
  }
]
```

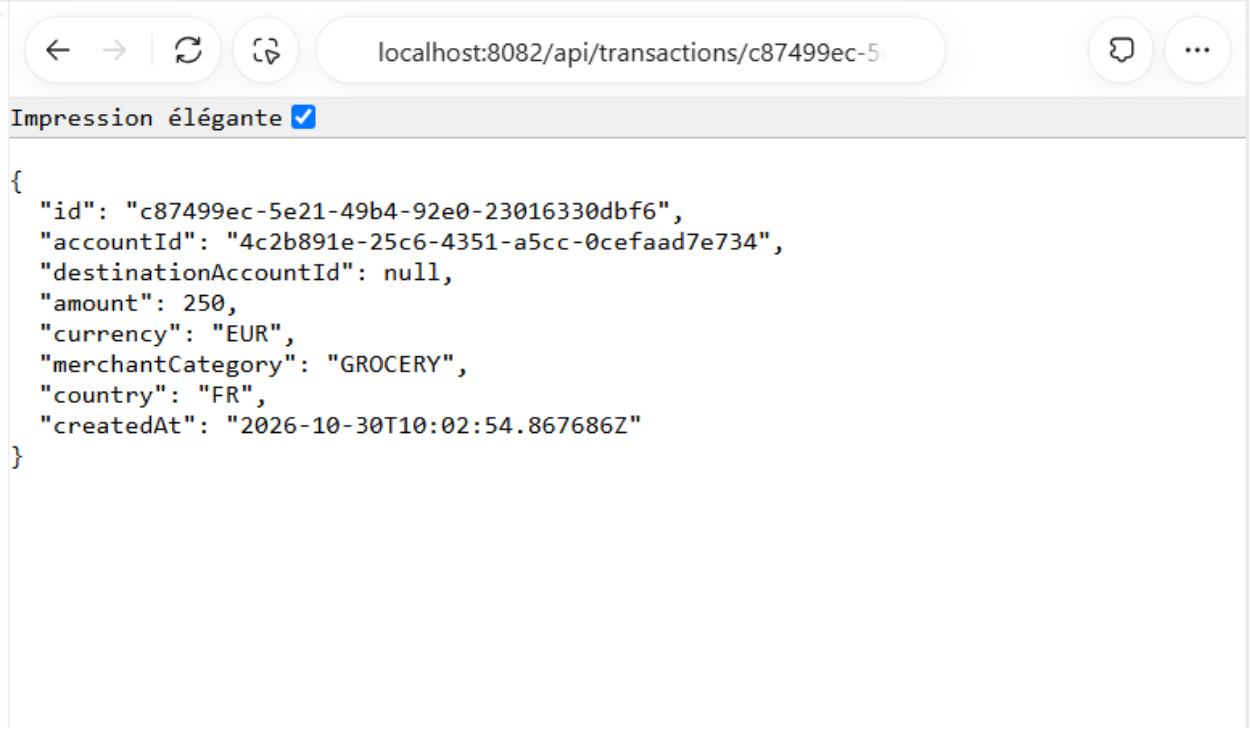
Capture - 02-02-solde-avant.png

Solde du compte source avant débit : 1500 EUR.



Capture - 02-03-transaction.png

Lecture de la transaction créée de 250 EUR ; ce GET ne montre pas le statut 201 du POST.



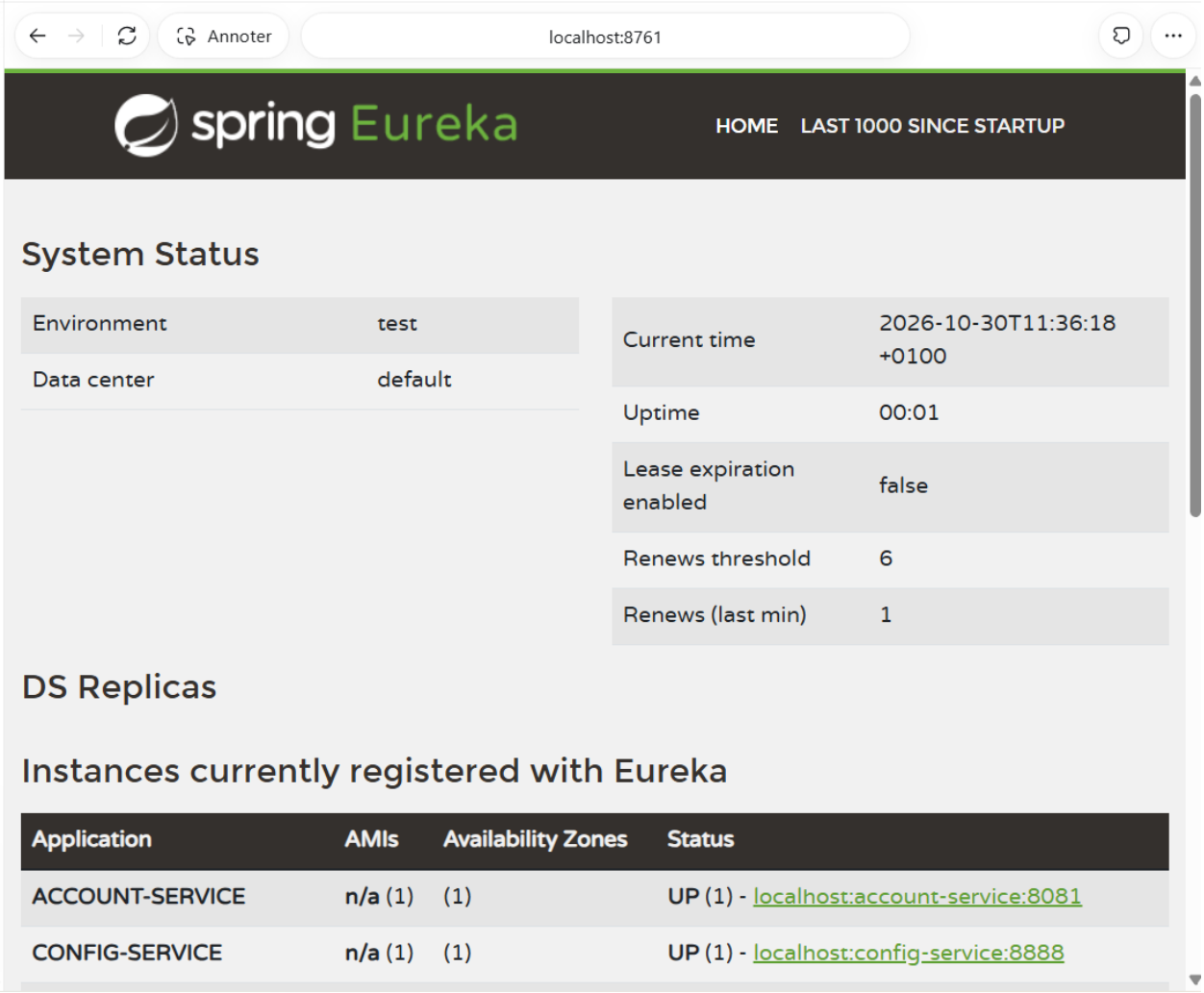
Capture - 02-04-solde-apres.png

Solde après le débit : 1250 EUR.



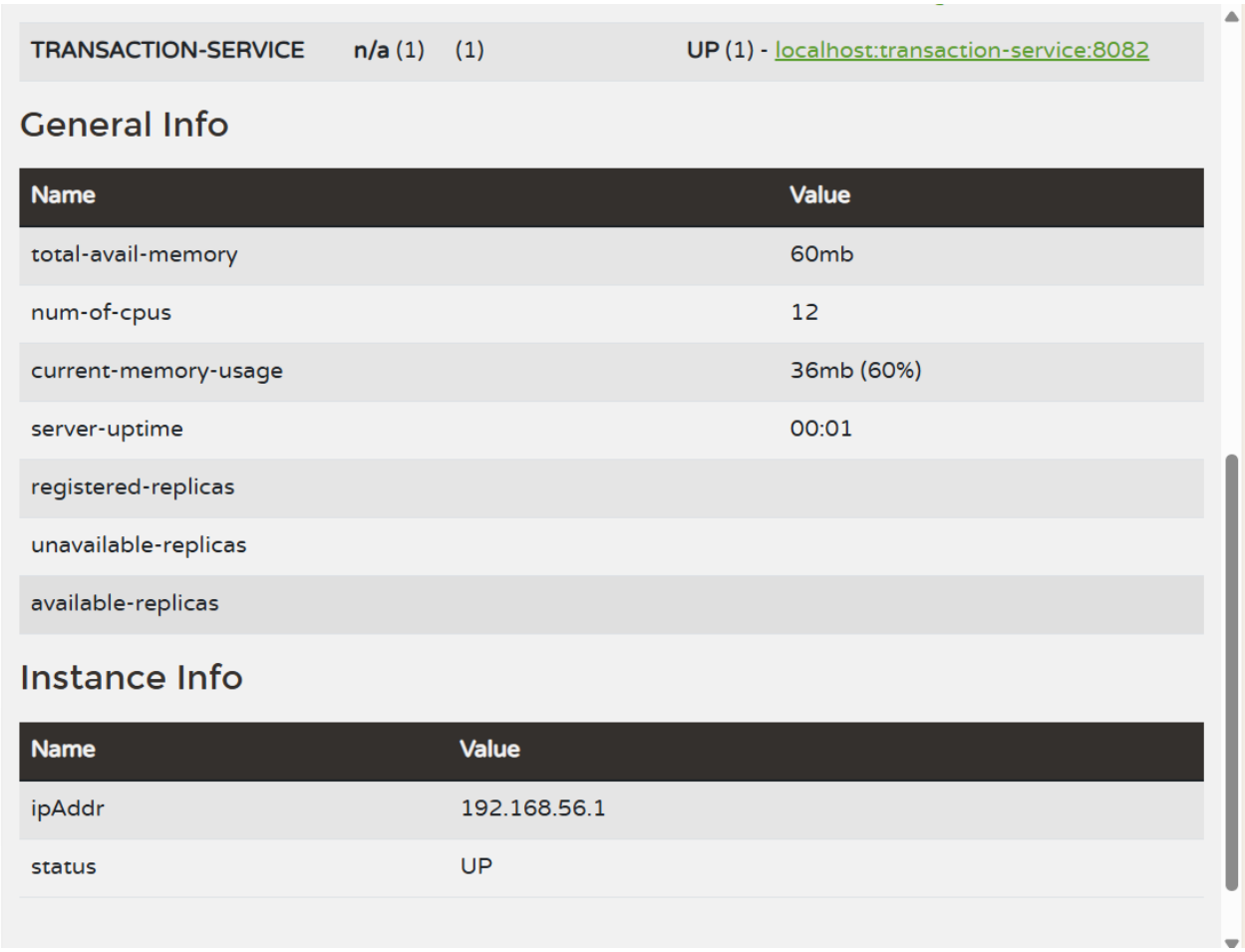
Capture - 03-01-eureka-a.png

Dashboard Eureka : account-service et config-service enregistrés et UP.



Capture - 03-01-eureka-b.png

Suite du dashboard : transaction-service également UP.



TRANSACTION-SERVICE	n/a (1)	(1)	UP (1) - localhost:transaction-service:8082

General Info

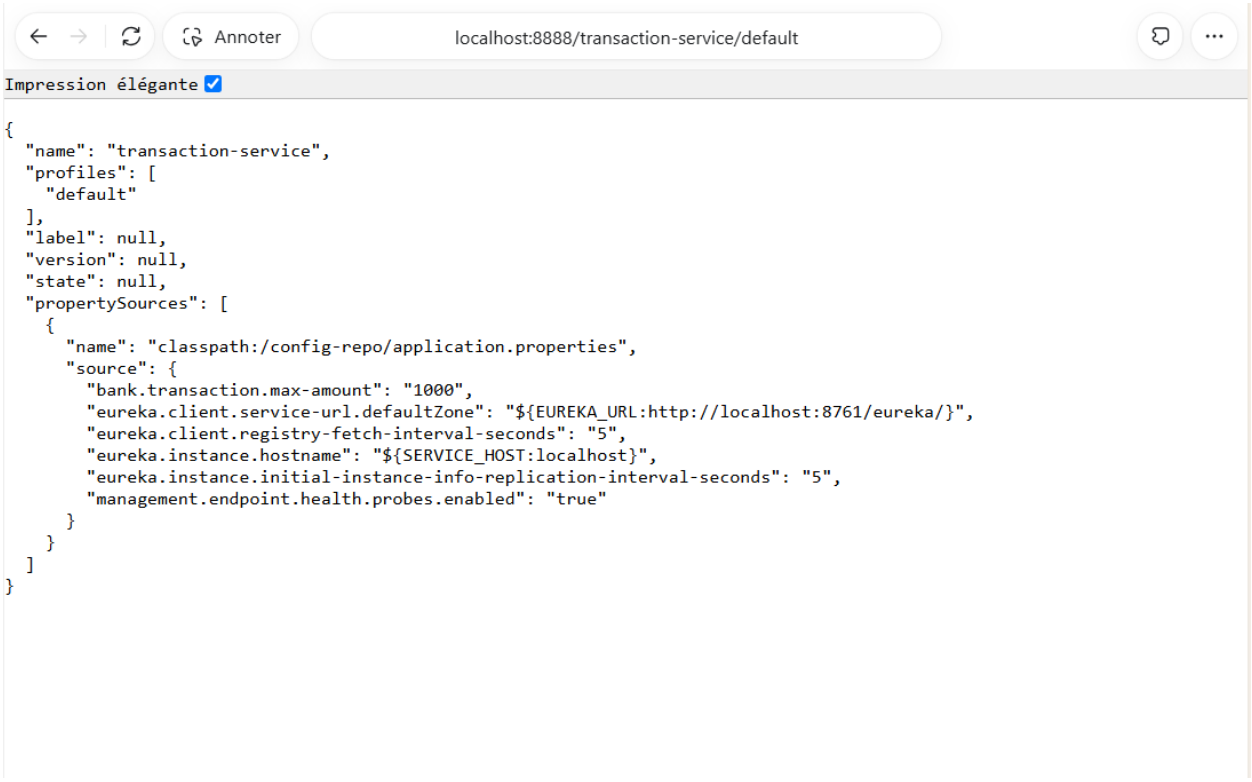
Name	Value
total-avail-memory	60mb
num-of-cpus	12
current-memory-usage	36mb (60%)
server-uptime	00:01
registered-replicas	
unavailable-replicas	
available-replicas	

Instance Info

Name	Value
ipAddr	192.168.56.1
status	UP

Capture - 03-02-config.png

Config Server : bank.transaction.max-amount=1000.




```
{
  "name": "transaction-service",
  "profiles": [
    "default"
  ],
  "label": null,
  "version": null,
  "state": null,
  "propertySources": [
    {
      "name": "classpath:/config-repo/application.properties",
      "source": {
        "bank.transaction.max-amount": "1000",
        "eureka.client.service-url.defaultZone": "${EUREKA_URL:http://localhost:8761/eureka/}",
        "eureka.client.registry-fetch-interval-seconds": "5",
        "eureka.instance.hostname": "${SERVICE_HOST:localhost}",
        "eureka.instance.initial-instance-info-replication-interval-seconds": "5",
        "management.endpoint.health.probes.enabled": "true"
      }
    }
  ]
}
```

Capture - 04-04-account-tools-a.png

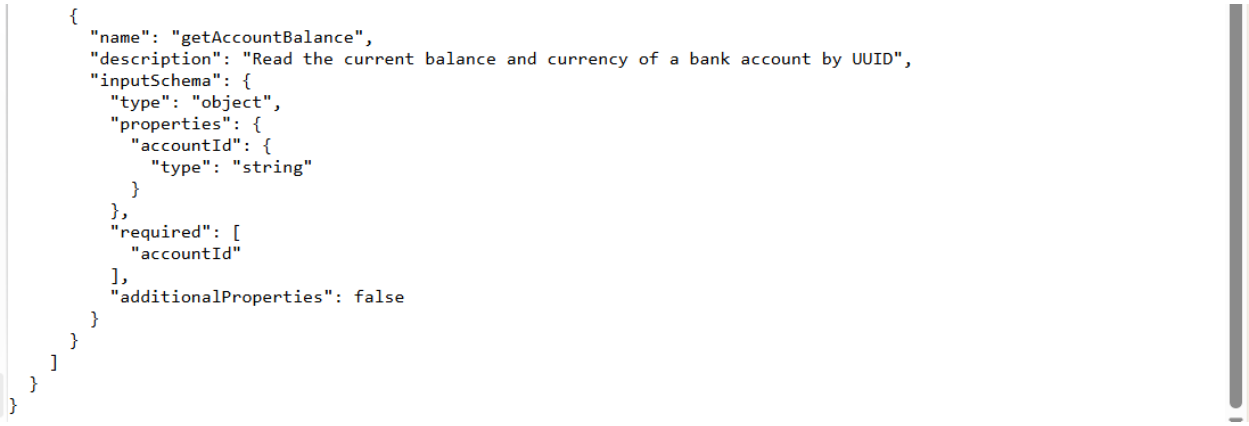
Réponse MCP enregistrée tools/list : outils du service des comptes.


```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "listAccounts",
        "description": "List all bank accounts and their identifiers",
        "inputSchema": {
          "type": "object",
          "properties": {

          },
          "required": [],
          "additionalProperties": false
        }
      },
      {
        "name": "getAccount",
        "description": "Get an existing account, its status, currency and balance by UUID",
        "inputSchema": {
          "type": "object",
          "properties": {
            "accountId": {
              "type": "string"
            }
          },
          "required": [
            "accountId"
          ],
          "additionalProperties": false
        }
      },
      {
        "name": "getAccountBalance",
        "description": "Read the current balance and currency of a bank account by UUID",
        "inputSchema": {
          "type": "object",
          "properties": {
            "accountId": {
              "type": "string"
            }
```

Capture - 04-04-account-tools-b.png

Suite du schéma de getAccountBalance ; accountId est requis.



```
      {
        "name": "getAccountBalance",
        "description": "Read the current balance and currency of a bank account by UUID",
        "inputSchema": {
          "type": "object",
          "properties": {
            "accountId": {
              "type": "string"
            }
          },
          "required": [
            "accountId"
          ],
          "additionalProperties": false
        }
      }
    ]
  }
}
```